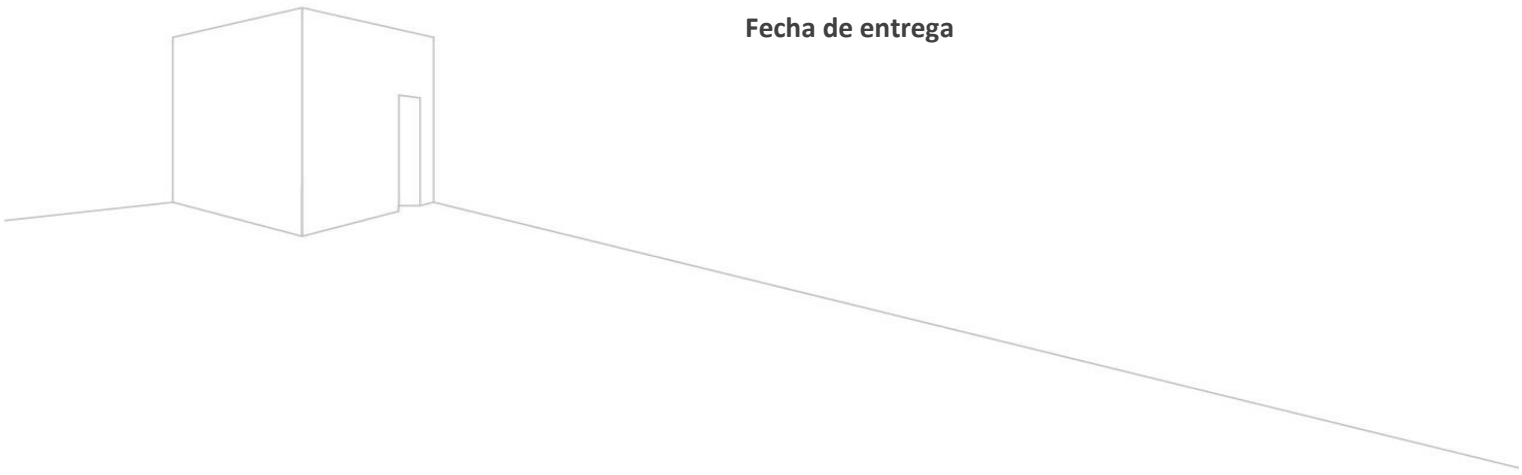
Evaluación II.

Asignatura: Tecnologías Aplicadas a Los Sistemas Inteligentes.

Nombre del docente: Osvaldo Javier Duarte Faúndez.

Nombre de los integrantes del grupo: Álvaro Villena, Felipe Martínez, Alan Escobar y Tomas Quezada

Fecha de entrega



Indice

Contenido

Indice 2

Introducción 3

Objetivos 4

Marco Teórico 5

Descripción del Proyecto TerraSense 9

Análisis Comparativo de los Modelos de Desarrollo 11

Modelo Seleccionado: Desarrollo Híbrido 13

Auditoría Tecnológica Aplicada a TerraSense 19

Ventajas, Desventajas y Gestión del Riesgo 20

Casos Reales de Referencia 22

Conclusiones 24

Referencias bibliográficas 25

Anexos 27

Introducción

Un sistema inteligente no es sólo un programa: es la unión de sensores, electrónica, firmware, comunicaciones y software que transforma una medición del mundo físico en una decisión. Cuando una parte de ese sistema es un equipo electrónico que se diseña, se fabrica y se lleva al campo, la manera en que se planifica su desarrollo pesa tanto como la tecnología elegida. Un error en una aplicación se corrige en minutos; un error en una placa de circuito impreso (PCB) obliga a rediseñarla, volver a fabricarla y esperar semanas antes de poder probar otra vez.

TerraSense es un instrumento portátil para medir el suelo que el equipo está desarrollando desde su origen. Una sonda industrial de siete variables se conecta a un microcontrolador ESP32 montado en una placa diseñada por el propio equipo; el equipo envía cada medición por Bluetooth de bajo consumo (BLE) a una aplicación móvil, que la interpreta según la etapa del cultivo incluso sin cobertura de internet, la guarda y la sincroniza con una base de datos en la nube que se administra desde un backoffice web.

El presente informe investiga los principales modelos de desarrollo tecnológico que se usan en la planificación estratégica (cascada, modelo en V, prototipado, espiral, metodologías ágiles y DevOps) y los contrasta con las condiciones reales de un proyecto electrónico construido desde cero. Con ese análisis se justifica el modelo adoptado en TerraSense: un **modelo híbrido** que combina la espiral guiada por riesgo y el prototipado evolutivo para el hardware, una forma de trabajo ágil para el software y prácticas DevOps como base común. También se explica por qué no se utilizó ninguno de esos modelos de manera pura.

El documento se organiza de la siguiente forma:

- **Marco teórico:** planificación tecnológica estratégica, modelos de desarrollo, auditoría tecnológica, factores de viabilidad, rol de los productos tecnológicos y entornos de implantación.
- **Caso aplicado:** descripción de TerraSense, comparación de los modelos frente a sus condiciones, justificación del modelo híbrido y aplicación de la auditoría tecnológica.
- **Evaluación:** ventajas, desventajas y gestión del riesgo técnico, contraste con casos reales de la industria y conclusiones.

Objetivos

Analizar los modelos de desarrollo tecnológico que se utilizan en la planificación estratégica y justificar, desde la perspectiva de un proyecto electrónico construido desde su origen, el modelo de desarrollo adoptado en el proyecto TerraSense.

Objetivos Específicos:

- Describir los conceptos de planificación tecnológica estratégica, auditoría tecnológica, viabilidad y entornos de implantación aplicados a los sistemas inteligentes.
- Caracterizar los modelos cascada, en V, prototipado, espiral, ágil y DevOps, identificando sus fortalezas y limitaciones.
- Comparar esos modelos frente a las condiciones propias de un equipo electrónico: costo de rehacer el hardware, tiempos de fabricación, incertidumbre de los requisitos e integración con el software.
- Fundamentar la combinación de modelos elegida para TerraSense y explicar por qué no se usó cada modelo de forma pura.
- Proponer instrumentos de auditoría y de verificación que acompañen cada etapa del desarrollo.

Marco Teórico

Planificación Tecnológica Estratégica:

La planificación tecnológica estratégica es el proceso con el que una organización decide qué tecnologías desarrollar o adoptar, en qué orden y con qué recursos, para que sus capacidades técnicas respondan a sus objetivos. No se limita a elegir componentes: define el camino de desarrollo, los momentos en que se revisa el avance y los criterios para seguir, corregir o detener un proyecto. En ese camino, el **modelo de desarrollo** es la herramienta que ordena el trabajo en el tiempo: establece qué se hace primero, cuándo se prueba, cómo se incorporan los cambios y quién decide (Pressman, 2010; Sommerville, 2011).

En un proyecto electrónico la planificación tiene una particularidad: conviven dos tipos de trabajo con costos de cambio muy distintos. El software se modifica, se prueba y se publica en horas, mientras que el hardware queda congelado cada vez que se manda a fabricar una placa o se compra un lote de componentes. Por eso la elección del modelo no es un trámite: define cuánto cuesta equivocarse y en qué momento se descubre el error.

Modelos de Desarrollo Tecnológico:

Un modelo de desarrollo, también llamado ciclo de vida, es una representación simplificada de las etapas por las que pasa un producto desde la idea hasta su operación, y de la forma en que esas etapas se encadenan. A continuación se describen los modelos más utilizados.

Modelo en cascada. Descrito por Royce (1970), organiza el desarrollo en fases secuenciales: requisitos, diseño, implementación, pruebas, operación y mantenimiento. Cada fase comienza cuando la anterior termina y se aprueba. Su fortaleza es el orden: un plan claro, documentación completa y un avance fácil de controlar. Su debilidad es que los errores de requisitos o de diseño se descubren recién en las pruebas, al final, cuando corregirlos es más caro. El propio Royce advirtió ese riesgo y recomendó construir una versión piloto antes de la definitiva.

Modelo en V. Es una variante de la cascada usada en ingeniería de sistemas y en la electrónica automotriz (Forsberg y Mooz, 1991). Cada etapa de diseño, en el lado izquierdo de la V, tiene asociada una etapa de prueba en el lado derecho: los requisitos se comprueban con pruebas del sistema completo, la arquitectura con pruebas de integración y el diseño detallado con pruebas de cada módulo. Su fortaleza es la trazabilidad entre lo que se pidió y la forma de comprobarlo; su debilidad es que sigue siendo secuencial y supone requisitos estables.

Prototipado. Consiste en construir versiones preliminares (montajes en protoboard, maquetas o pantallas de prueba) para aclarar los requisitos y probar ideas antes del diseño definitivo. El prototipo puede ser desechable o evolutivo: en el segundo caso se refina hasta convertirse en el producto (Pressman, 2010). Reduce la incertidumbre de forma rápida y barata e involucra al usuario, pero sin control el prototipo se transforma en producto sin documentación ni pruebas suficientes.

Modelo en espiral. Propuesto por Boehm (1988), avanza en vueltas sucesivas. Cada vuelta recorre cuatro cuadrantes: fijar objetivos, alternativas y restricciones; analizar las alternativas e identificar y resolver los riesgos; desarrollar y verificar el producto de esa vuelta; y planificar la siguiente. El riesgo guía el orden del trabajo: se ataca primero lo que más puede hacer fracasar el proyecto, y cada vuelta termina en una revisión donde se decide si se continúa. Es adecuado cuando la incertidumbre técnica y el costo de un error son altos, aunque exige experiencia para identificar los riesgos y más esfuerzo de gestión.

Metodologías ágiles. Nacen del Manifiesto Ágil (Beck et al., 2001), que valora a las personas y su interacción, el producto funcionando, la colaboración con el cliente y la respuesta al cambio. Scrum organiza el trabajo en iteraciones de una a cuatro semanas, con roles y eventos definidos (Schwaber y Sutherland, 2020); Kanban hace visible el trabajo en un tablero y limita las tareas en curso, con un flujo continuo sin iteraciones fijas (Anderson, 2010). Su fortaleza es la adaptación y la retroalimentación rápida; su supuesto es que el producto puede cambiarse barato en cada iteración, algo cierto en el software pero no en una placa que demora semanas en fabricarse.

DevOps. Une el desarrollo y la operación mediante cultura y automatización: control de versiones, integración continua (cada cambio se prueba automáticamente), entrega continua y monitoreo del producto en uso (Humble y Farley, 2010; Kim et al., 2016). Asegura una calidad constante y publicaciones frecuentes y seguras. Está pensado para productos digitales: no sirve para fabricar hardware, pero sí alcanza al firmware cuando el equipo puede actualizarse a distancia.

Auditoría Tecnológica:

La auditoría tecnológica es una evaluación sistemática y documentada del estado de las tecnologías de un proyecto u organización: qué tecnologías se usan, cómo están implementadas, qué riesgos presentan y si cumplen lo que se espera de ellas. Sus funciones son diagnosticar, detectar brechas, verificar el cumplimiento de requisitos y respaldar las decisiones. Entre sus instrumentos se cuentan las listas de verificación, las revisiones de diseño, el inventario tecnológico, las matrices de evaluación, las pruebas documentadas y la observación del uso real. En un proyecto en desarrollo la auditoría no es un evento al final, sino una revisión que acompaña el cierre de cada etapa.

Factores Clave de Viabilidad:

Un plan tecnológico es viable cuando puede llevarse a cabo y sostenerse en el tiempo. Para un sistema inteligente con hardware propio se consideran los siguientes factores:

- **Viabilidad técnica:** existen los componentes, las herramientas y el conocimiento para construir el sistema y lograr que sus partes se comuniquen de forma confiable.
- **Viabilidad operativa:** el usuario puede usar el sistema en su entorno real, con su nivel de conocimiento y sin depender de condiciones que no controla.
- **Viabilidad de recursos:** el equipo dispone de las personas, el tiempo, el instrumental de laboratorio y el presupuesto que exige cada etapa, incluidas las fabricaciones de placas.
- **Viabilidad del entorno y normativa:** el equipo opera en bandas de radio de uso libre y resguarda los datos de los usuarios, como la ubicación de cada medición.
- **Sostenibilidad del soporte:** es posible mantener, reparar y actualizar el equipo una vez entregado.

Rol de los Productos Tecnológicos en los Sistemas Inteligentes:

Un sistema inteligente percibe, procesa, decide e informa o actúa. Los productos tecnológicos hacen posible cada uno de esos pasos: los sensores convierten fenómenos físicos en señales, los microcontroladores las digitalizan y las transmiten, y el software las interpreta. Lee (2008) denomina sistemas ciberfísicos a esta integración de computación, redes y procesos físicos, y advierte que su diseño exige considerar a la vez el comportamiento físico y el digital. La calidad de la decisión depende del eslabón más débil: una lectura errónea del sensor produce una mala recomendación, por buena que sea la aplicación.

Entornos de Implantación:

- **Entorno físico:** campo abierto, con humedad, polvo, radiación solar, temperaturas extremas, golpes y la inserción repetida de la sonda en el suelo. El equipo funciona con batería, por lo que el consumo de energía es una restricción de diseño.
- **Entorno digital:** muchos campos no tienen cobertura celular; el enlace Bluetooth es de corto alcance; los teléfonos de los usuarios son de distintos modelos y la sincronización con la nube ocurre cuando vuelve la conexión.
- **Entorno organizacional:** agricultores y asesores con distinta familiaridad con la tecnología, que necesitan indicaciones en lenguaje simple y un soporte técnico capaz de resolver problemas a distancia.

Descripción del Proyecto TerraSense

Problema y Propuesta:

Las decisiones de riego y fertilización suelen tomarse por apreciación visual o a partir de análisis de laboratorio que tardan días y se pagan por muestra. TerraSense propone medir en el momento y en cualquier punto del campo, y obtener de inmediato una interpretación de la lectura según la etapa del cultivo, aunque no haya internet.

El Sistema Visto desde la Raíz:

Visto como un proyecto electrónico que se concibe desde cero, TerraSense se compone de los bloques que muestra la Figura 1:

- **Sonda de suelo:** sensor industrial de siete variables (humedad, temperatura, conductividad eléctrica, pH, nitrógeno, fósforo y potasio) que se comunica por un bus RS-485 con el protocolo Modbus, un estándar de la instrumentación industrial (Modbus Organization, 2006).
- **Electrónica de control:** un ESP32, microcontrolador con Bluetooth integrado, en una placa diseñada por el equipo. La placa incluye el transceptor RS-485, un transistor MOSFET que energiza la sonda sólo durante la medición para ahorrar batería, la medición de la batería, un sensor ambiental de temperatura, humedad y presión conectado por I2C, tres LED, un buzzer y un botón de vinculación.
- **Firmware:** el programa del ESP32, que lee los sensores, arma la trama de datos, gestiona la vinculación con el teléfono y maneja los indicadores luminosos y sonoros.
- **Enlace inalámbrico:** Bluetooth de bajo consumo con una trama única de 20 bytes que lleva el suelo, el ambiente y la batería.
- **Aplicación móvil:** recibe y valida la trama, la interpreta con un motor agronómico que funciona en el teléfono sin internet, la guarda con su ubicación y la sincroniza cuando hay conexión.
- **Nube y backoffice:** una base de datos con permisos por usuario y un sitio web de soporte para administrar los equipos y publicar nuevas versiones del firmware.

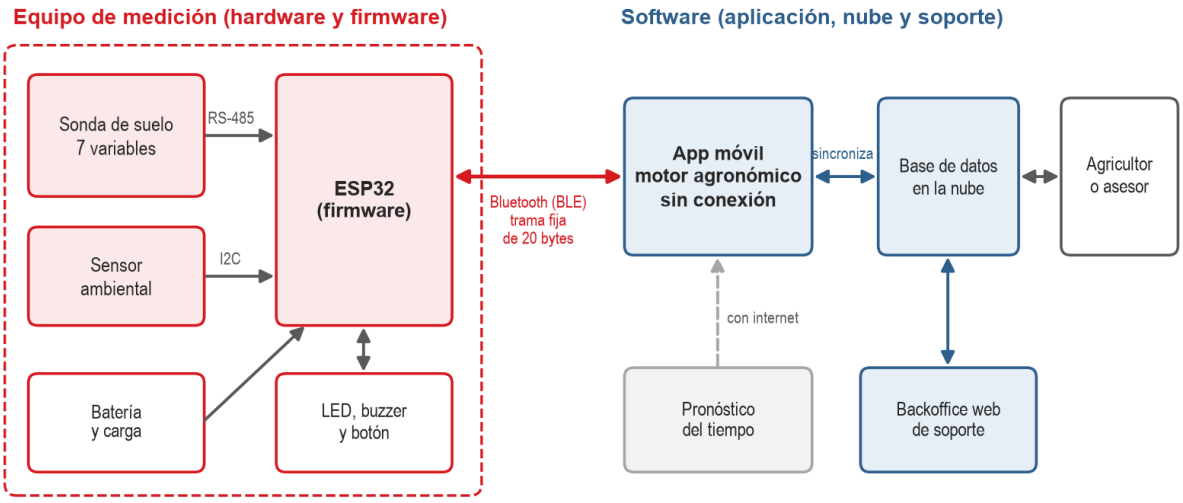


Figura 1. Arquitectura en bloques del sistema TerraSense.

Condiciones que Determinan la Planificación:

Cuatro características del proyecto condicionan la elección del modelo de desarrollo:

- **Dos naturalezas distintas:** el hardware es caro y lento de cambiar; el software, barato y rápido.
- **Alta incertidumbre técnica inicial:** al comenzar no se sabía si la sonda respondería como indicaba su manual, cuánto alcance tendría el enlace Bluetooth, cuánto duraría la batería ni qué tan confiables serían las lecturas en un suelo real.
- **Requisitos agronómicos que se afinan con el uso:** los criterios para interpretar una lectura y la forma de presentarla al agricultor mejoran a medida que se prueban.
- **Un equipo pequeño** de estudiantes, con tiempo, instrumental y presupuesto acotados, que trabaja a la vez en hardware y software.

Análisis Comparativo de los Modelos de Desarrollo

A partir de las condiciones descritas se definieron siete criterios de comparación. La Tabla 1 resume cómo responde cada modelo frente a ellos cuando se aplica a un proyecto electrónico.

Criterio	Cascada	En V	Prototipado	Espiral	Ágil	DevOps
Manejo de la incertidumbre técnica inicial	Baja	Baja	Alta	Muy alta	Media	Baja
Costo de corregir un error de hardware	Muy alto: se detecta al final	Alto	Bajo en etapas tempranas	Bajo: se detecta en la vuelta	Alto si se rehace la placa en cada iteración	No aplica
Adaptación a requisitos que cambian	Muy baja	Muy baja	Alta	Media	Muy alta	Alta
Trazabilidad entre requisitos y pruebas	Media	Muy alta	Baja	Alta	Media	Alta
Velocidad de entrega del software	Baja	Baja	Media	Media	Muy alta	Muy alta
Respeto por los tiempos de fabricación y compra	Sí	Sí	Parcial	Sí: cada vuelta es una fabricación	No	No aplica
Carga de gestión para un equipo pequeño	Media	Alta	Baja	Alta	Baja	Media

Tabla 1. Comparación de modelos según las condiciones de un proyecto electrónico.

Ningún modelo obtiene la mejor respuesta en todos los criterios. Los modelos secuenciales (cascada y en V) ordenan bien el trabajo y la verificación, pero descubren tarde los errores; los iterativos de software (ágil y DevOps) responden muy bien al cambio, pero no consideran los tiempos de fabricación; la espiral y el prototipado son los que mejor reducen la incertidumbre física, a cambio de más gestión o de menos disciplina.

Por qué no se Utilizó Ningún Modelo de Forma Pura:

Cascada pura. Habría obligado a cerrar todos los requisitos y el diseño de la placa antes de saber si la sonda respondía, cuánto alcanzaba el Bluetooth o cuánto duraba la batería. Un ejemplo concreto: en la primera prueba en banco, el módulo ambiental comprado resultó ser una variante sin sensor de humedad. En un desarrollo secuencial ese hallazgo habría aparecido en las pruebas finales, con placas ya fabricadas. Además, los criterios agronómicos sólo se afinan usando el sistema, algo que la cascada no admite.

Modelo en V puro. Su lógica de asociar una prueba a cada nivel de diseño se adoptó, pero su secuencia de una sola pasada no: supone que la arquitectura y la placa se diseñan bien a la primera, lo que no ocurre en un primer producto.

Prototipado por sí solo. Fue indispensable al inicio, pero sin análisis de riesgo ni revisiones con decisión se corre el riesgo de pasar directamente de un montaje que funciona en el escritorio a fabricar, sin verificar consumo, ruido eléctrico o comportamiento en el campo.

Espiral para todo el proyecto. Es la adecuada para el hardware, pero aplicarla con el mismo rigor a cada pantalla de la aplicación sería una carga de gestión sin beneficio, porque esos cambios son baratos y reversibles.

Ágil puro (Scrum). Una iteración de dos semanas no alcanza para diseñar, fabricar, recibir y armar una placa, y entregar un incremento de hardware funcionando en cada iteración no es realista ni barato para un equipo estudiantil. Las empresas que llevan el trabajo ágil al hardware lo logran con simulación intensiva y arquitecturas modulares a gran escala (Furuhjelm et al., 2017), infraestructura que este proyecto no tiene.

DevOps por sí solo. No es un modelo para planificar un producto, sino una práctica para entregarlo y operarlo: no indica cómo diseñar una placa ni en qué orden atacar los riesgos técnicos.

Modelo Seleccionado: Desarrollo Híbrido

Por lo anterior, TerraSense se desarrolla con un **modelo híbrido** que asigna a cada tipo de trabajo el modelo que mejor responde a su costo de cambio, y que une ambas partes mediante un contrato de interfaz definido al inicio. La Figura 2 muestra su estructura: una pista de hardware y firmware que avanza en vueltas de espiral con prototipos evolutivos, una pista de software que avanza en iteraciones ágiles, el contrato de 20 bytes entre ambas y una base común de prácticas DevOps.

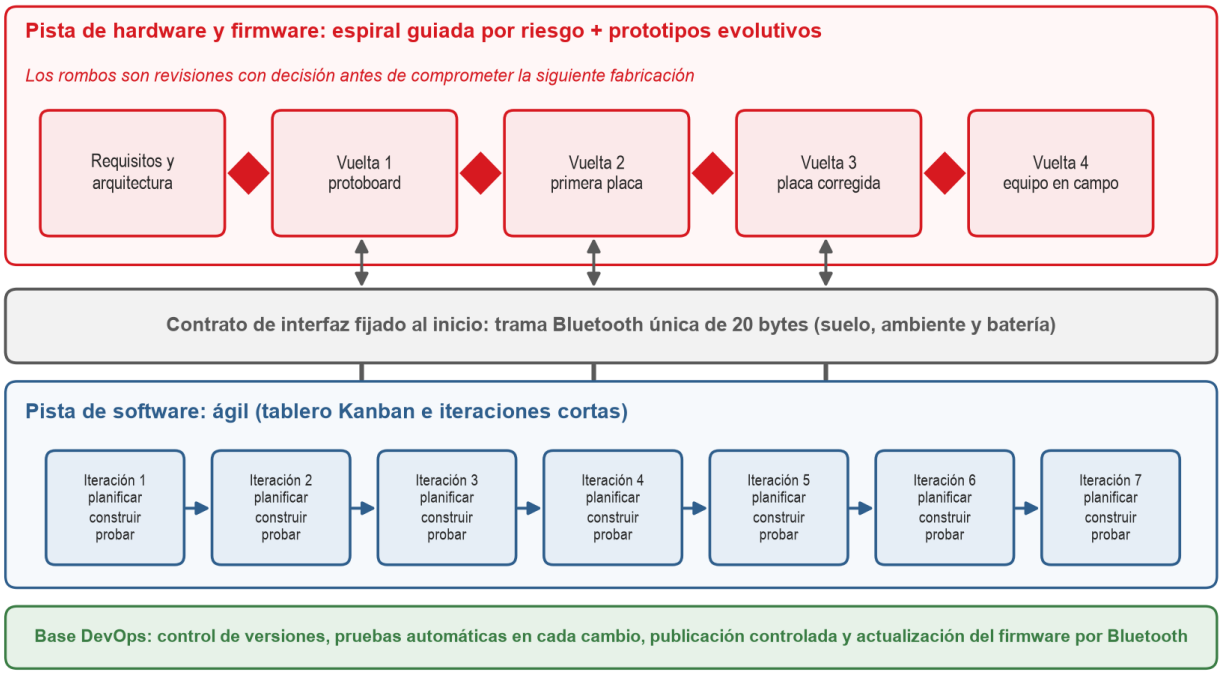


Figura 2. Modelo híbrido de desarrollo de TerraSense.

Eje Rector: Espiral Guiada por el Riesgo:

El proyecto se planifica en vueltas. Cada una se define por su objetivo, el riesgo principal que ataca, el prototipo que produce y la forma de comprobarlo, y termina en una revisión donde el equipo decide si se compromete la siguiente fabricación (Figura 3). Así, el dinero y el tiempo se invierten en orden creciente: primero lo barato que despeja la mayor duda y al final lo caro, cuando el riesgo ya es bajo.

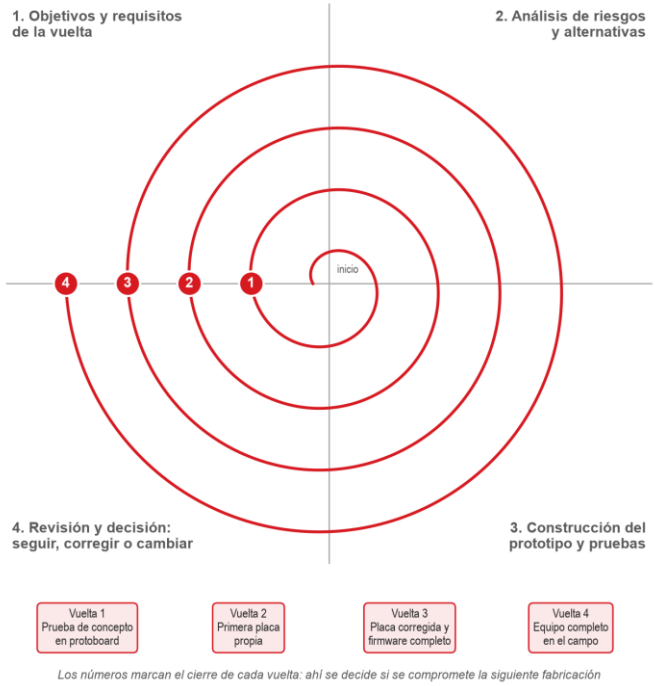


Figura 3. Vueltas de la espiral de desarrollo del equipo.

Vuelta	Objetivo	Riesgo que se ataca	Prototipo	Cómo se comprueba
1. Prueba de concepto	Comprobar que las piezas clave se comunican.	Que la sonda no responda por RS-485 o que el teléfono no reciba los datos por Bluetooth.	ESP32 de desarrollo, transceptor RS-485 y sensores montados en protoboard, alimentados por USB.	Lecturas en el monitor serie validadas con su código de control (CRC) y conexión con la aplicación.
2. Primera placa propia	Llevar el circuito a una placa diseñada por el equipo.	Errores de conexión entre pines, ruido en el bus y consumo excesivo.	Esquemático y placa diseñados en KiCad.	Revisión del diseño con reglas eléctricas y coincidencia de pines entre el esquemático y el firmware.
3. Placa corregida y firmware completo	Incorporar lo aprendido y completar las funciones del equipo.	Comunicación inestable con la sonda, batería mal medida y vinculación confusa.	Placa fabricada con bus polarizado, MOSFET de la sonda, medición de batería, LED, buzzer y botón; firmware con vinculación y trama de 20 bytes.	Pruebas en banco con un menú de mantenimiento que ejercita cada sensor y salida por separado, y pruebas con la aplicación.
4. Equipo completo en el campo	Validar el equipo como producto.	Lecturas poco confiables en suelo real, alcance, autonomía, resistencia del gabinete y comprensión del usuario.	Equipo con gabinete y batería, reposo profundo y actualización del firmware por Bluetooth.	Mediciones en agua, tierra húmeda y soluciones de referencia, y jornadas de uso con agricultores.

Tabla 2. Vueltas de la espiral en el desarrollo de TerraSense.

Las dos primeras vueltas están cerradas. La tercera está en curso: la placa ya fue fabricada y el firmware se prueba en banco, donde la sonda entregó su primera lectura válida. La cuarta vuelta es la que lleva el equipo al campo.

Prototipado Evolutivo dentro de Cada Vuelta:

Los prototipos no se desechan: cada uno es la base del siguiente. El firmware escrito para el montaje en protoboard es la base del firmware de la placa, y el menú de pruebas que se creó para revisar los prototipos quedó como herramienta de mantenimiento del producto. Los hallazgos de cada prototipo se transformaron en decisiones de diseño:

- La primera respuesta de la sonda llegó con el código de control dañado porque la línea que habilita la transmisión se liberaba antes del último bit; se corrigió en el firmware antes de diseñar la placa.
- Las lecturas mostraban bytes sueltos producidos por el bus flotando al cambiar de transmisión a recepción; la placa incorporó resistencias de polarización del bus.
- El módulo ambiental resultó ser una variante sin humedad; se especificó el sensor correcto y el firmware ahora identifica el modelo conectado y lo informa al arrancar.

Verificación por Niveles, Tomada del Modelo en V:

Del modelo en V se adoptó la regla de que cada nivel de diseño tiene su prueba asociada (Figura 4). Los módulos se prueban por separado, la placa se prueba en banco, el equipo se prueba completo junto a la aplicación y, al final, se prueba en el campo con usuarios. Esta verificación no ocurre una sola vez, sino al cierre de cada vuelta, y su detalle se presenta en el Anexo B.

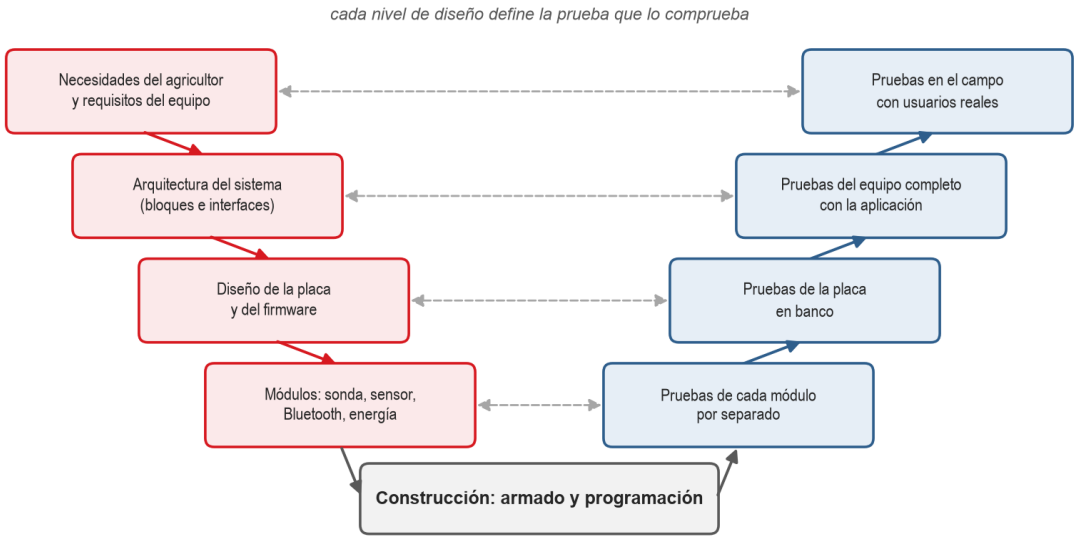


Figura 4. Verificación por niveles aplicada al equipo.

Contrato de Interfaz: el Punto de Unión:

Para que el software no tuviera que esperar al hardware, al inicio se fijó la trama Bluetooth: un único mensaje de 20 bytes con las variables de suelo, las ambientales y la batería, más un byte de versión. Cuando un dato no está disponible viaja con un valor reservado, de modo que la aplicación nunca inventa una lectura. Con ese contrato la pista de software pudo construir y probar la recepción con tramas de ejemplo mientras la placa todavía estaba en diseño; por ejemplo, la aplicación rechaza una trama sin lectura de suelo y no guarda nada, comportamiento comprobado con pruebas automáticas antes de integrar la sonda.

Pista de Software: Trabajo Ágil:

La aplicación, la base de datos y el backoffice avanzan en iteraciones cortas de una a dos semanas, con un tablero Kanban (por hacer, en curso, en revisión y terminado). Se prefirió Kanban a un Scrum estricto porque el mismo equipo atiende el hardware: los roles y eventos formales de Scrum serían una carga para un equipo pequeño, y un flujo continuo absorbe mejor las interrupciones que provoca la llegada de una placa o de un componente. De Scrum se conservaron el objetivo de cada iteración, la revisión de lo construido y la reflexión sobre cómo mejorar. Así se entregaron, en incrementos, la vinculación del equipo, la captura de la medición, el motor agronómico por etapa del cultivo, el historial, la sincronización y el backoffice de soporte.

Base Común: Prácticas DevOps:

- **Control de versiones** de todo el proyecto: firmware, aplicación, backoffice, base de datos y diseño de la placa.
- **Integración continua:** cada cambio ejecuta automáticamente las pruebas de la aplicación y del backoffice y una revisión de la documentación; si una prueba falla, el cambio se corrige antes de continuar.
- **Cambios de la base de datos versionados**, de modo que cualquier copia del sistema puede reconstruirse igual a la de producción.
- **Pruebas del firmware en el computador** para la lógica que no depende del hardware, como los comandos de mantenimiento y las secuencias de los indicadores.
- **Actualización del firmware por Bluetooth:** el backoffice ya carga y publica nuevas versiones y el firmware se está preparando para recibirlas, lo que extiende DevOps a los equipos que ya estén en manos de los usuarios.

Modelo	Qué se toma	Dónde se aplica	Qué se deja fuera
Espiral	Vueltas guiadas por el riesgo y revisión con decisión al cierre.	Todo el proyecto, con mayor rigor en el hardware.	El análisis formal de riesgo para cambios menores de software.
Prototipado	Prototipos evolutivos que se refinan hasta el producto.	Protoboard, primera placa, placa corregida y equipo completo.	Los prototipos desechables sin documentación.
Modelo en V	Una prueba asociada a cada nivel de diseño.	Verificación del hardware y del firmware.	La secuencia rígida de una sola pasada.
Ágil (Kanban con prácticas de Scrum)	Tablero, iteraciones cortas, revisión y mejora continua.	Aplicación, base de datos y backoffice.	Roles y eventos formales completos; iteraciones aplicadas a la fabricación de placas.
DevOps	Versiones, pruebas automáticas, publicación controlada y actualización remota.	Todo el software y el firmware.	Nada relevante: es la base común de las dos pistas.
Cascada	Documentar requisitos y diseño antes de comprometer una fabricación.	El cierre de cada vuelta de hardware.	La secuencia única de principio a fin.

Tabla 3. Aporte de cada modelo al modelo híbrido.

Auditoría Tecnológica Aplicada a TerraSense

En TerraSense la auditoría tecnológica se aplica como una revisión al cierre de cada vuelta de la espiral y, en el software, de forma continua mediante las pruebas automáticas. La Tabla 4 resume los ámbitos revisados, los instrumentos usados y lo que cada revisión aportó al proyecto.

Ámbito	Qué se revisa	Instrumento	Hallazgo o acción
Diseño electrónico	Coincidencia de pines entre esquemático y firmware, reglas eléctricas y polarización del bus.	Revisión de diseño con lista de verificación.	Se agregaron resistencias de polarización al bus RS-485 y se fijó un único archivo de pines que debe coincidir con el esquemático.
Componentes	Que el componente recibido sea el especificado.	Inspección y prueba en banco con lectura del identificador del chip.	El módulo ambiental era una variante sin humedad: se especificó el sensor correcto y el firmware informa el modelo detectado.
Firmware	Compilación sin advertencias, uso de memoria y lógica de las funciones.	Compilación con todas las advertencias activas y pruebas en el computador.	El programa ocupa el 89 % del espacio disponible: la distribución de la memoria debe decidirse antes de habilitar la actualización por Bluetooth.
Comunicación y seguridad	Cifrado del enlace y proceso de vinculación.	Revisión de seguridad del enlace Bluetooth.	La vinculación exige mantener el botón tres segundos; el enlace aún no está cifrado y se registra como mejora de la siguiente vuelta.
Software y datos	Pruebas automáticas y permisos de acceso a la base de datos.	Integración continua y revisión de las reglas de acceso.	Cada usuario sólo accede a sus propias mediciones y los cambios de la base de datos quedan versionados.
Operación en el campo	Autonomía, alcance, resistencia y comprensión de las recomendaciones.	Pruebas en el campo y observación de usuarios.	Planificado para la cuarta vuelta.

Tabla 4. Auditoría tecnológica aplicada a TerraSense.

Esta auditoría también sostiene la viabilidad del proyecto. La viabilidad técnica quedó respaldada en las primeras vueltas, cuando la sonda, el sensor ambiental y el enlace Bluetooth funcionaron juntos; la operativa se apoya en el motor agronómico sin conexión y en el lenguaje simple de las recomendaciones; la de recursos, en que cada fabricación se compromete sólo después de una revisión, y la sostenibilidad del soporte, en el backoffice y la actualización remota del firmware.

Ventajas, Desventajas y Gestión del Riesgo

Gestión del Riesgo Técnico:

El modelo híbrido se eligió, ante todo, para controlar el riesgo técnico. La Tabla 5 relaciona los riesgos principales del proyecto con la respuesta que el modelo les da.

Riesgo	Probabilidad	Impacto	Respuesta del modelo
La sonda no responde o entrega datos corruptos.	Media	Alto	Primera vuelta dedicada a la comunicación; cada lectura se valida con su código de control y existe un menú de diagnóstico del bus.
Error de diseño en una placa fabricada.	Media	Alto	Prototipo previo en protoboard y revisión de diseño antes de cada fabricación.
La batería se agota antes de terminar una jornada.	Media	Alto	MOSFET que apaga la sonda entre mediciones, medición de batería y reposo profundo planificado.
Un componente distinto al especificado.	Media	Medio	Inspección en banco; el firmware detecta e informa el modelo conectado.
La aplicación y el equipo no se entienden.	Baja	Alto	Contrato de 20 bytes con byte de versión y pruebas automáticas de la decodificación.
El agricultor no comprende la recomendación.	Media	Medio	Iteraciones ágiles con retroalimentación de usuarios y lenguaje simple.
Una publicación de software introduce errores.	Baja	Medio	Integración continua: sólo se publica lo que aprobó todas las pruebas automáticas.

Tabla 5. Riesgos técnicos y respuesta del modelo híbrido.

Ventajas del Modelo Híbrido:

- Los errores de hardware se descubren en la vuelta más barata posible, antes de comprometer una fabricación.
- El software avanza sin esperar al hardware gracias al contrato de interfaz.
- Cada pista trabaja al ritmo que le corresponde: semanas por vuelta en el hardware y días por cambio en el software.
- Las pruebas automáticas mantienen la calidad aunque el software cambie con frecuencia.
- La actualización remota del firmware permite corregir y mejorar los equipos ya entregados.

Desventajas del Modelo Híbrido:

- Exige coordinar dos ritmos de trabajo distintos y mantener al día el contrato que los une.
- Requiere más disciplina de documentación y de revisión que un modelo único.
- La pista de software puede adelantarse al hardware real: algo probado con tramas de ejemplo debe volver a comprobarse con el equipo.
- La identificación de riesgos depende de la experiencia del equipo, que en un proyecto estudiantil todavía se está formando.

Casos Reales de Referencia

La combinación de modelos no es una solución exclusiva de este proyecto. Los siguientes casos documentados muestran cómo la industria resuelve el mismo problema de desarrollar hardware y software a la vez.

Saab, avión de combate Gripen E. Saab aplica prácticas ágiles en todas las disciplinas del programa, incluido el hardware, con más de cien equipos coordinados en iteraciones comunes de tres semanas (Furuhjelm et al., 2017). Lo logra gracias a una arquitectura modular con interfaces estables y a una fuerte inversión en simulación. La lección para TerraSense es que el trabajo ágil puede alcanzar al hardware cuando las interfaces están fijas; el contrato de 20 bytes cumple ese papel a pequeña escala, pero sin la infraestructura de simulación de Saab el hardware sigue necesitando la espiral.

John Deere, maquinaria agrícola. El grupo de soluciones inteligentes de John Deere construyó una metodología ágil escalada que combina Scrum, el marco ágil escalado, programación extrema y principios Lean, y que en 2012 ya practicaban más de 1.500 personas (InfoQ, 2012). Muestra que, incluso en el sector agrícola y a gran escala, se combinan marcos de trabajo en lugar de aplicar uno solo.

Fabricantes con el modelo Agile–Stage-Gate. Cooper y Sommer (2016, 2018) estudiaron a fabricantes de productos físicos que mantienen compuertas de decisión entre etapas y, dentro de cada etapa, trabajan en iteraciones ágiles. Informan una respuesta más rápida a los cambios del cliente y mayor productividad, junto con dificultades como el escepticismo inicial de la dirección y la necesidad de equipos dedicados. Las revisiones con decisión al cierre de cada vuelta de TerraSense cumplen el papel de esas compuertas.

Industria automotriz y el modelo en V. La norma ISO 26262 de seguridad funcional organiza el desarrollo de la electrónica de los vehículos con un ciclo en V en los niveles de sistema, hardware y software, y exige que cada requisito tenga su verificación asociada (International Organization for Standardization, 2018). De ahí proviene la verificación por niveles que TerraSense aplica a su equipo.

Caso	Sector	Combinación usada	Lección para TerraSense
Saab Gripen E	Aeronáutica	Ágil en hardware y software con interfaces modulares y simulación.	Las interfaces fijas permiten avanzar en paralelo.
John Deere	Maquinaria agrícola	Scrum, marco ágil escalado, programación extrema y Lean.	En el agro también se combinan marcos de trabajo.
Fabricantes Agile–Stage-Gate	Productos manufacturados	Compuertas de decisión con iteraciones ágiles dentro de cada etapa.	Revisión con decisión antes de cada fabricación.
Industria automotriz	Electrónica vehicular	Modelo en V exigido por la norma ISO 26262.	Una prueba asociada a cada nivel de diseño.

Tabla 6. Síntesis de los casos de referencia.

Conclusiones

La investigación muestra que los modelos de desarrollo no compiten entre sí en abstracto: cada uno responde a un tipo de trabajo. La cascada y el modelo en V ordenan y verifican, el prototipado y la espiral reducen la incertidumbre, y las metodologías ágiles y DevOps permiten cambiar rápido sin perder calidad. En un sistema inteligente con hardware propio esos tipos de trabajo conviven, por lo que elegir un único modelo obliga a renunciar a algo importante.

En TerraSense, la diferencia entre el costo de cambiar una placa y el de cambiar una aplicación fue el argumento central. Por eso el hardware y el firmware avanzan en una espiral guiada por el riesgo, con prototipos evolutivos y una prueba asociada a cada nivel de diseño, mientras el software avanza en iteraciones ágiles sostenidas por prácticas DevOps. El contrato de 20 bytes, fijado al inicio, es lo que permite que ambas pistas trabajen en paralelo sin bloquearse.

El proceso ya mostró su valor: errores como el código de control dañado, el ruido del bus o el sensor ambiental equivocado se detectaron en prototipos baratos, antes de comprometer una fabricación. La cascada habría descubierto esos problemas al final, y un enfoque ágil puro habría intentado rehacer placas al ritmo de iteraciones que el hardware no puede seguir.

Los casos de Saab, John Deere, los fabricantes que usan Agile–Stage-Gate y la industria automotriz confirman que la industria combina modelos en lugar de aplicar uno solo. Para proyectos educativos y emprendimientos tecnológicos se recomienda: definir temprano las interfaces entre hardware y software; ordenar el trabajo físico según sus riesgos y probar lo barato antes de fabricar; automatizar las pruebas del software desde el primer día, y realizar revisiones con decisión al cierre de cada etapa. La tarea que queda es la cuarta vuelta, que llevará el equipo al campo y pondrá a prueba el modelo con usuarios reales.

Referencias bibliográficas

- Anderson, D. J. (2010). Kanban: Successful evolutionary change for your technology business. Blue Hole Press.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J. y Thomas, D. (2001). Manifiesto por el desarrollo ágil de software. https://agilemanifesto.org/iso/es/manifesto.html
- Boehm, B. W. (1988). A spiral model of software development and enhancement. Computer, 21(5), 61–72. https://doi.org/10.1109/2.59
- Cooper, R. G. y Sommer, A. F. (2016). The Agile–Stage-Gate hybrid model: A promising new approach and a new research opportunity. Journal of Product Innovation Management, 33(5), 513–526. https://doi.org/10.1111/jpim.12314
- Cooper, R. G. y Sommer, A. F. (2018). Agile–Stage-Gate for manufacturers: Changing the way new products are developed. Research-Technology Management, 61(2), 17–26.
- Forsberg, K. y Mooz, H. (1991). The relationship of system engineering to the project cycle. INCOSE International Symposium, 1(1), 57–65.
- Furuhjelm, J., Segertoft, J., Justice, J. y Sutherland, J. J. (2017). Owning the sky with agile. Scrum Inc. https://archive.appliedframeworks.com/wp-content/uploads/2020/10/Owning-the-Sky-with-Agile.pdf
- Humble, J. y Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.
- InfoQ. (2012). [Entrevista a Chad Holdorf sobre la metodología ágil escalada del grupo de soluciones inteligentes de John Deere]. https://www.infoq.com/news/2012/03/deere
- International Organization for Standardization. (2018). Road vehicles: Functional safety (ISO 26262:2018).
- Kim, G., Humble, J., Debois, P. y Willis, J. (2016). The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations. IT Revolution Press.
- Lee, E. A. (2008). Cyber physical systems: Design challenges. En 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (pp. 363–369). IEEE. https://doi.org/10.1109/ISORC.2008.25

- Modbus Organization. (2006). MODBUS over serial line: Specification and implementation guide V1.02. https://modbus.org/specs.php

- Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

- Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 1–9.

- Schwaber, K. y Sutherland, J. (2020). La guía de Scrum. https://scrumguides.org

Anexos

Anexo A. Matriz de Decisión Metodológica:

Cada modelo se evaluó de 1 (muy bajo) a 5 (muy alto) en seis criterios ponderados según su importancia para un proyecto electrónico construido desde cero. El puntaje final es la suma de cada nota multiplicada por su ponderación.

Modelo	Control del riesgo en el hardware (25 %)	Capacidad de absorber cambios (20 %)	Bajo costo de rehacer lo fabricado (15 %)	Rapidez de entrega del software (15 %)	Verificación y trazabilidad (15 %)	Ajuste a un equipo pequeño (10 %)	Puntaje
Cascada	2	1	2	1	4	3	2,05
En V	3	1	2	1	5	2	2,35
Prototipado	3	4	3	3	2	4	3,15
Espiral	5	3	4	2	4	2	3,55
Ágil	2	5	2	5	3	4	3,40
DevOps	1	4	1	5	4	3	2,85
Híbrido	5	4	4	5	5	3	4,45

Tabla 7. Matriz de decisión ponderada.

El modelo híbrido obtiene el mayor puntaje porque combina la nota alta de la espiral en el control del riesgo físico con la de los modelos ágiles y DevOps en la entrega del software. Su única nota intermedia es el ajuste a un equipo pequeño, que corresponde al esfuerzo de coordinar las dos pistas.

Anexo B. Protocolo de Verificación por Niveles:

Nivel	Qué se prueba	Ejemplo en TerraSense	Criterio de aprobación
Módulo	Cada componente por separado.	Sonda en banco consultada por RS-485; sensor ambiental por I2C; LED y buzzer.	Respuesta con código de control válido y valores coherentes con el ambiente.
Placa en banco	La placa armada con todas sus funciones.	Menú de mantenimiento: medición de batería, salidas, diagnóstico del bus I2C y barrido del bus RS-485.	Cada prueba entrega el resultado esperado y las tensiones están dentro de rango.
Equipo completo con la aplicación	La integración entre el equipo y el teléfono.	Vinculación con el botón, medición y recepción de la trama de 20 bytes.	La aplicación guarda una medición completa y rechaza, sin guardar nada, una trama sin lectura de suelo.
Campo	El equipo en su entorno real.	Mediciones en agua, tierra húmeda y soluciones de referencia; alcance, autonomía y uso por agricultores.	Lecturas coherentes con la referencia, una jornada completa con una carga y recomendaciones comprendidas.
Software (continuo)	Cada cambio en la aplicación y el backoffice.	Pruebas automáticas en la integración continua.	Todas las pruebas pasan antes de integrar o publicar.

Tabla 8. Protocolo de verificación por niveles.

Anexo C. Glosario:

- **BLE (Bluetooth Low Energy):** variante de Bluetooth de bajo consumo, pensada para enviar datos breves entre un equipo y un teléfono.
- **CRC (código de redundancia cíclica):** valor que acompaña a un mensaje y permite detectar si llegó alterado.
- **ESP32:** microcontrolador con Wi-Fi y Bluetooth integrados; es el cerebro del equipo.
- **Firmware:** programa que corre dentro del microcontrolador y controla el hardware.
- **I2C:** bus de comunicación de dos hilos para conectar sensores dentro de una placa.
- **Integración continua:** práctica en que cada cambio del código se prueba automáticamente antes de unirse al proyecto.
- **Modbus:** protocolo de comunicación industrial con el que la sonda entrega sus lecturas.
- **MOSFET:** transistor que funciona como interruptor electrónico; en TerraSense enciende la sonda sólo al medir.
- **PCB (placa de circuito impreso):** placa donde se montan y conectan los componentes electrónicos.
- **RS-485:** estándar eléctrico para transmitir datos por cable de forma robusta frente al ruido, común en sensores industriales.